

Universidade Federal do Maranhão
Centro de Ciências Exatas e Tecnologias
Engenharia da Computação

Bruno Feres de Souza

Disciplina: Métodos Formais

Código: EEC0019

18 de março de 2026

Conteúdo programático

- **Noções de Engenharia de Software.**
- Formalismo no desenvolvimento de software.
- Técnicas para especificação, verificação e validação formais.
- Linguagens de especificação formal.
- *Model cheking* e provadores de teoremas.

Sumário

- Introdução
- O que é Engenharia de Software?
- Desafios na Engenharia de Software
- Processos de software e ciclos de vida
- Inspeções de Software
- Gerenciamento de Projetos de Software
- Modelo CMMI
- Métodos Formais

Contextualização

- Nas décadas de 50 e 60, havia o entendimento de que falhas são inerentes ao software.
 - O código final sempre será repleto de falhas.
 - A etapa de codificação deve terminar o quanto antes para corrigir os defeitos logo.
 - A modelagem ocorre à medida que se codifica.
- A chamada "Abordagem das Hordas Mongóis" era muito difundida à época.
 - Utilização de muitos programadores inexperientes para o desenvolvimento dos sistemas, ao invés de um grupo qualificado de pessoas.

Contextualização

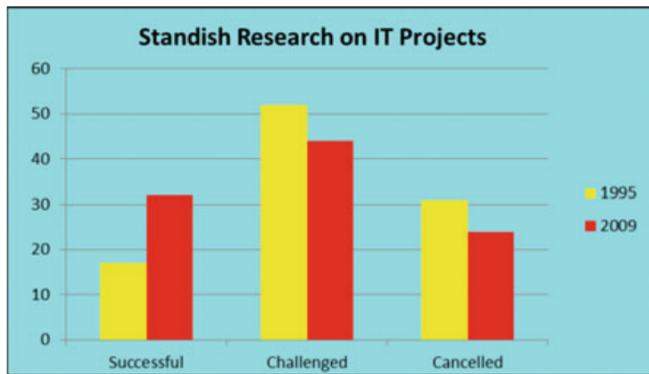
- No final de 1960, o termo "**crise de software**" foi utilizado pela primeira vez, em duas conferências organizadas pela OTAN para discutir o estado atual do desenvolvimento de software.
 - Problemas com orçamento.
 - Prazos estourados.
 - Baixa confiabilidade.
 - Qualidade a desejar.

Contextualização

- Nas conferências organizadas pela OTAN, a Engenharia de Software surgiu como disciplina. Isto se deveu aos seguintes fatores:
 - Reconhecimento de que programação é diferente de ciência e de matemática.
 - Percepção de que programadores são como engenheiros, ao criar produtos de software.
 - A engenharia trata da modelagem e construção de produtos seguros.
 - A matemática é importante para a engenharia clássica e deve ser também para a desenvolvimento de software.
 - Engenheiros têm uma responsabilidade profissional em assegurar que seus produtos são construídos apropriadamente.

Contextualização

- Para ilustrar o estado de desenvolvimento de software, o grupo Standish estudou a extensão de problemas com projetos de informática entre os anos 90 e 2000.



Definição

- Segundo a IEEE, a **Engenharia de Software** pode ser assim definida:

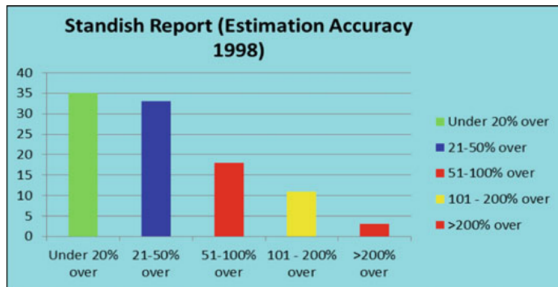
Engenharia de Software é a aplicação de uma abordagem sistemática, disciplinada e quantificável para o desenvolvimento, operação e manutenção de software, isto é, corresponde à aplicação de princípios de engenharia ao desenvolvimento de software, e ao estudo de tais abordagens.

Definição

- A Engenharia de Software inclui:
 - 1 O emprego de metodologias para projetar, desenvolver e testar software para atender às necessidades dos usuários.
 - 2 O uso de princípios de engenharia no contexto de software.
 - 3 A observância a critérios de qualidade e segurança do software.
 - 4 O uso de matemática para auxiliar no projeto e na verificação de produtos de software.
 - 5 Boas práticas de projeto e de gerenciamento.
 - 6 Atenção à manutenção e ao suporte.

Desafios

- Os principais desafios na Engenharia de Software incluem a entrega de software de alta qualidade no prazo e no orçamento.
- Para dimensionar tais grandezas, é necessário a realização de medições e estimativas, que por vezes são imprecisas. Por exemplo, o grupo Standish estudou estimativas de custo de projetos e obteve os seguintes resultados:



Desafios

- De fato, para tornar o desenvolvimento de software mais efetivo é necessário observar os seguintes pontos:
 - A gerência do projeto precisa determinar quão bom é seu processo de estimativa (de custo, pessoal, prazo) e realizar as melhorias adequadas.
 - Para tanto, é importante considerar métricas de software.
 - Abordagens de gerenciamento de risco são importantes para antever problemas e mitigar a extensão dos impactos .
 - A qualidade do software deve ser planejada para possibilitar a entrega de um produto adequado. O custo de retrabalho não é negligenciável.

Definição

- Um **processo de software** refere-se ao conjunto de atividades relacionadas que levam à produção de um produto de software. Geralmente, tais processos incluem sub-processos:
 - Processo para gerenciamento
 - Processo para aquisição de requisitos
 - Processo para desenvolvimento
 - Processo para codificação
 - Processo para teste
 - Processo para medições
 - Processo para melhorias
 - etc

Definição

- O(s) processo(s) utilizado(s) depende(m) do tamanho, da complexidade e da natureza da empresa de software.
- Para ser efetivo e impactar positivamente a qualidade do software produzido, o processo deve ser amplamente utilizado na empresa. Isso se materializa pelos seguintes ativos organizacionais:
 - Política formal de desenvolvimento de software.
 - Mapas de processo com os fluxos das atividades.
 - Guias e procedimentos para descrever todos os processos em detalhes.
 - Listas de controle para ajudar no desempenho dos processos.
 - Modelos para atividades específicas (por exemplo: projeto, teste, etc).
 - Material de treinamento.

Definição

- Associado aos processos, existem os **ciclos de vida de software**, que consistem de diversas fases. Dentre outros, três ciclos são bem estabelecidos¹:
 - Modelo em cascata.
 - Modelo incremental.
 - Modelo baseado em reuso.
- A escolha de um determinado ciclo é definida pelas idiossincracias de cada organização e de cada projeto, podendo haver inclusive a combinação de abordagens.

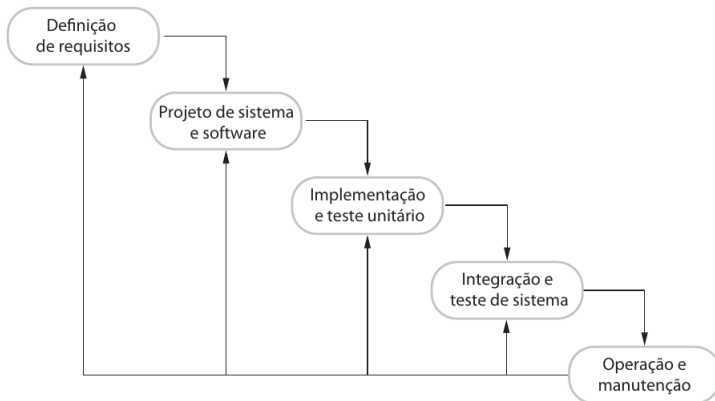
¹Esta seção foi baseada no Capítulo 2 do livro: SOMMERVILLE, I. Engenharia de Software. 9ª Edição. São Paulo, Brasil: Pearson, 2011.

Modelo em cascata

- O **Modelo em cascata** tem sido amplamente utilizado em cenários onde os requisitos do sistema são conhecido a priori ou podem ser identificados em fases iniciais do desenvolvimento.
- Assim, neste modelo, as atividades podem ser planejadas antes de sua realização de fato.

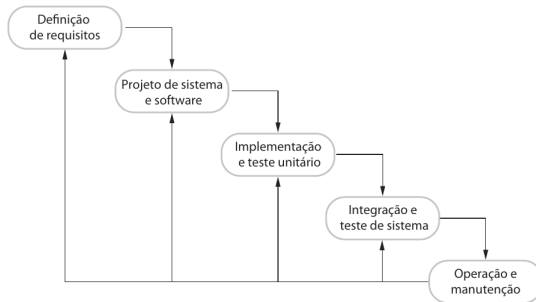
Modelo em cascata

- De maneira genérica, o modelo consiste do encadeamento de fases de desenvolvimento, com início e fim bem definidos. Tais fases são ilustradas abaixo:



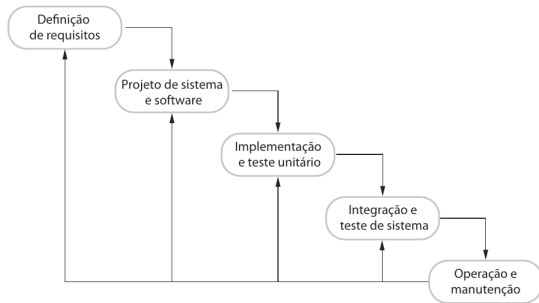
Modelo em cascata

- Na fase de **definição de requisitos**, os serviços, restrições e metas são estabelecidos por meio de consulta aos usuários.
- Após o entendimento dos requisitos, os detalhes são discutidos e uma especificação do sistema é acordada.



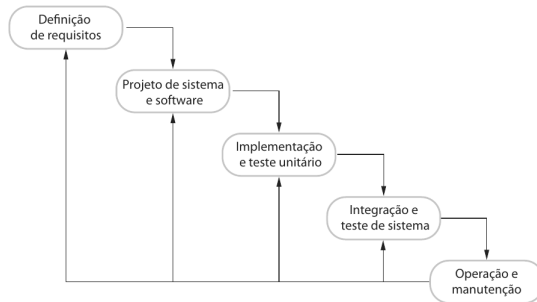
Modelo em cascata

- Na fase de **projeto de sistema e software**, é definida uma arquitetura geral do sistema, estabelecendo que recursos de hardware e software serão empregados.
- Nesta fase, tem-se a identificação e a descrição das abstrações fundamentais do sistema, e seus relacionamentos.



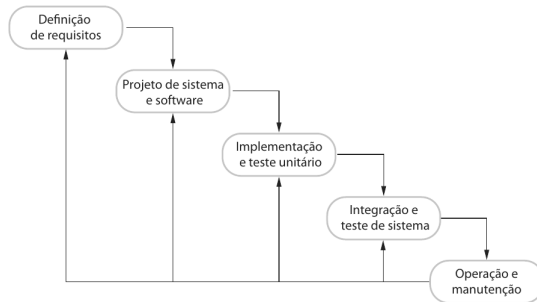
Modelo em cascata

- Na fase de **implementação e teste unitário**, o projeto do software é desenvolvido como um conjunto de programas ou unidades de programas.
- Nesta fase, é verificado se cada unidade atende a sua especificação.



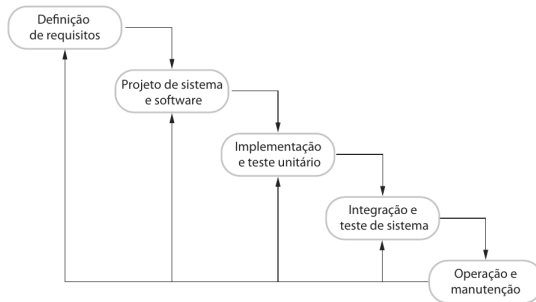
Modelo em cascata

- Na fase de **integração e teste de sistema**, os programas ou unidades de programas são integrados e testados como um sistema completo para assegurar que os requisitos do software tenham sido atendidos.
- Após o teste, o software é entregue ao cliente.



Modelo em cascata

- Na fase de **operação e manutenção**, o sistema é instalado e colocado em uso.
- A manutenção envolve a correção de erros não detectados previamente, a melhoria da implementação feita e a ampliação de funcionalidades.

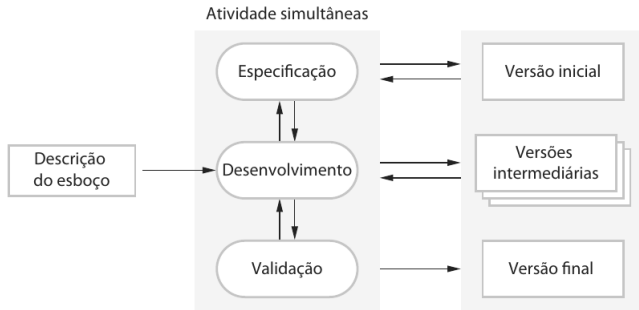


Modelo incremental

- O **Modelo incremental** é baseado na ideia de desenvolver uma implementação inicial, expô-la aos comentários do cliente e continuar por meio de versões até que o sistema esteja adequado.
- Este modelo é melhor que a abordagem em cascata para sistemas em que os sistemas não são inteiramente compreendidos a priori e os requisitos podem se alterar bastante.

Modelo incremental

- Neste modelo, as atividades de especificações, desenvolvimento e validação são intercaladas e há constante interação entre elas. O modelo é genericamente ilustrado abaixo:



Modelo incremental

- No modelo incremental, cada versão incorpora alguma funcionalidade necessária ao cliente. Geralmente, as funcionalidades mais urgentes são introduzidas antes.
- Isso permite três vantagens em relação ao modelo em cascata:
 - ① É mais fácil obter retorno dos clientes sobre o desenvolvimento que foi feito.
 - ② O custo de acomodar as mudanças nos requisitos do cliente é reduzido.
 - ③ É possível obter entrega e implementação rápida de um software útil ao cliente.

Modelo incremental

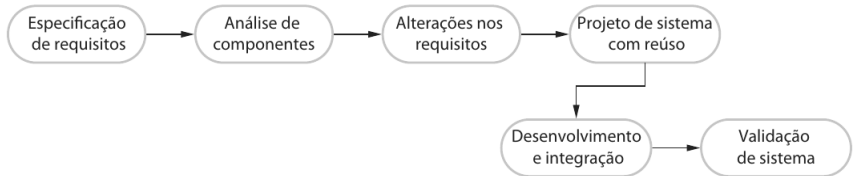
- Em contrapartida, a flexibilidade do modelo incremental impõe as seguintes dificuldades, do ponto de vista gerencial:
 - 1 O processo não é tão visível, pois dependendo da agilidade do desenvolvimento, pode não ser viável a confecção de documentos de acompanhamento a cada versão.
 - 2 O software pode se alterar muito no decorrer do desenvolvimento, tornando difícil a acomodação de futuras mudanças

Modelo baseado em reuso

- O **reuso de componentes** de software é prática informal e corriqueira. Projetos e códigos semelhantes são frequentemente buscados, modificados e incorporados ao desenvolvimento de sistemas.
- Nos últimos anos, vários processos de desenvolvimento de software tem focado no reuso sistemático. Como isso, viu-se a necessidade de uma ampla base de componentes e de *frameworks* de integração desses componentes.
- Exemplos de componentes para reuso incluem *web services*, objetos em alguma linguagem e sistemas individuais.

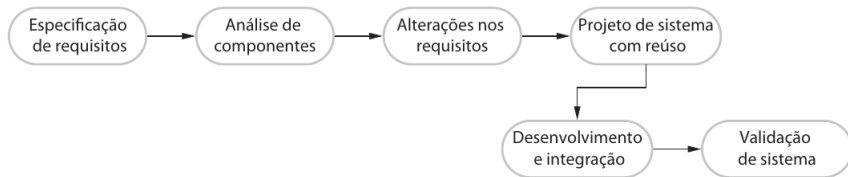
Modelo baseado em reuso

- O **modelo baseado em reuso** é genericamente ilustrado abaixo. Nele, as fases de especificação de requisitos e validação de sistema são como vistas anteriormente.



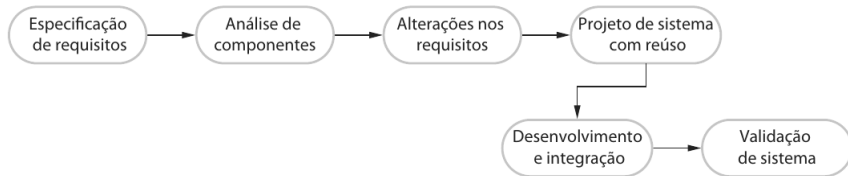
Modelo baseado em reuso

- Na fase de **análise de componentes**, dada a especificação de requisitos, é feita uma busca por componentes que satisfaçam a especificação.
- Na busca por componentes adequados, pode não haver correspondência exata, exigindo algum tipo de adaptação.



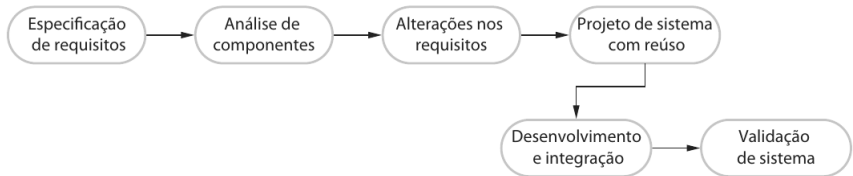
Modelo baseado em reuso

- Na fase de **alteração nos requisitos**, os requisitos podem ser alterados de acordo com a disponibilidade de componentes.
- Com a alteração dos requisitos, novas buscas por componentes podem ser realizadas.



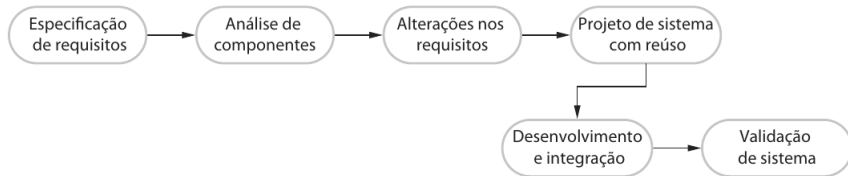
Modelo baseado em reuso

- Na fase de **projeto de sistema com reuso**, o *framework* do sistema é projetado, levando com conta os componentes disponíveis.
- Em caso de componentes essenciais não disponíveis, software novo pode ser desenvolvido.



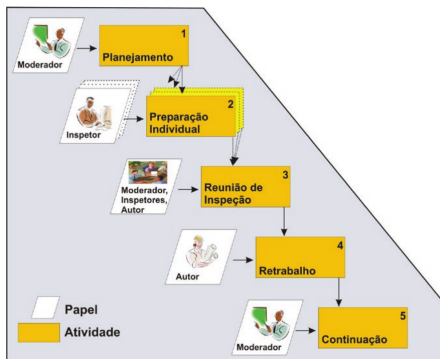
Modelo baseado em reuso

- Na fase de **desenvolvimento e integração**, software próprio é desenvolvido e integrado com componentes de reuso.
- A integração de todos os componentes pode fazer parte do processo de desenvolvimento.



Inspeções de Software

- **Inspeções de Software** consistem em formas detalhadas de revisar os artefatos construídos. O seu intuito é garantir a qualidade do software.
- Há várias abordagens para conduzir tais inspeções. Nos anos 70, Fagan desenvolveu² o processo ilustrado abaixo:



²Veja o artigo KALINOWSKI, M.; SPÍNOLA, R.O. Introdução à Inspeção de Software. Engenharia de Software Magazine, 1:68-74, 2007.

Inspeções de Software

- As atividades do processo de Fagan são descritas abaixo:
 - **Planejamento:** um moderador define o contexto da inspeção, seleciona os inspetores e distribui o material a ser inspecionado.
 - **Preparação:** os inspetores estudam os artefatos em busca de discrepâncias.
 - **Reunião:** o moderador, os inspetores e os autores dos artefatos se reúnem para discutir os achados e o moderador decide se são defeitos ou falso positivos.
 - **Retrabalho:** os autores corrigem os defeitos apontados pelos inspetores e confirmados pelo moderador.
 - **Continuação:** o material corrigido é analisado pelo moderador, que decide pela adequação ou não no mesmo. Em caso negativo, uma nova inspeção pode ser requisitada.

Gerenciamento de Projetos de Software

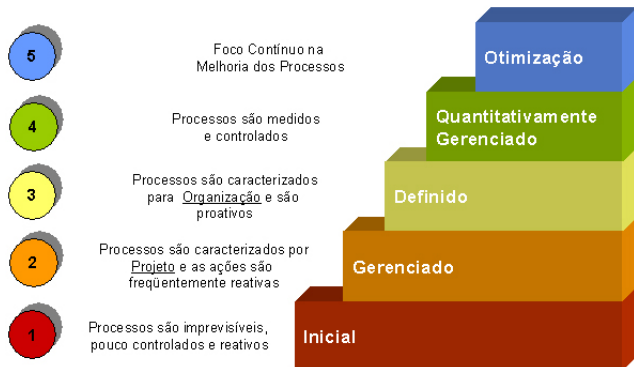
- O **gerenciamento de projetos** de software buscam fornecer o cenário adequado para a construção de um software de qualidade.
- Comumente, ele envolve as seguintes atividades:
 - Estimativa de custo, de pessoal e de prazos para o projeto
 - Identificação e gerenciamento de riscos.
 - Preparação do plano de projeto
 - Preparação de um cronograma e marcos a serem observados.
 - Obtenção do plano e do cronograma.
 - Designação de pessoal para o projeto.
 - Monitoramento de progresso, custo, pessoas, riscos, qualidade, prazos, etc do projeto.
 - Tomada de ações corretivas.
 - Atualização de planos e cronograma.
 - Comunicação do andamento do projeto às partes interessadas.
 - Preparação de relatórios e apresentações.

Modelo CMMI

- O **modelo CMMI** (*Capability Maturity Model Integration*) corresponde a um programa para ajudar as empresas a implementar as melhores práticas em engenharia de software e de sistemas.
- Ele foi desenvolvido para SEI (*Software Engineering Institute*) da Universidade Carnegie Mellon como um modelo para mensurar a maturidade de organizações em desenvolver software.

Modelo CMMI

- O modelo CMMI consiste de cinco níveis de maturidade, e cada nível foca na implementação de práticas relacionadas a determinadas áreas do processo de desenvolvimento de software. Os níveis são ilustrados a seguir:



Métodos Formais

- Dijkstra e Hoare argumentaram que a maneira correta de se desenvolver um software é derivar um programa a partir de suas especificações utilizando técnica matemáticas e, em seguida, aplicar provas rigorosas para demonstrar que o programa obtido está de acordo com o que foi especificado.
- O termo **Métodos Formais** refere-se à descrição formal de uma linguagem de especificação e a um método para projetar e implementar sistemas de computador.
- Estes serão os objetos de estudo desta disciplina.

Bibliografia

- ① O'REGAN, G. Concise Guide to Formal Methods: Theory, fundamentals and industry applications. 1ª Edição. Cham, Suíça: Springer, 2017.
 - Capítulo 1.
- ② SOMMERVILLE, I. Engenharia de Software. 9ª Edição. São Paulo, Brasil: Pearson, 2011.
 - Capítulo 2.

Dúvidas?